FinQuest: A Gamified Financial Literacy Platform for Grade 11–12 Students

Vanguard India Challenge — Milestone 2 Progress Report

Abstract

FinQuest teaches financial literacy through reels, manga-style chapters, quizzes, fastest-finger duels, a "survive the month" simulation and a moderated forum. Waves 0–2 of our build plan are implemented on `main`: all six feature modules, a 185-question bank, 13 chapters and 12 reel slots. The build, type check, lint and 48 unit tests pass. A review found one critical and several high-severity bugs, which are scheduled first. The database is not yet deployed to Supabase, so no live, browser or student tests have run.

1 Current Approach and Changes since M1

The stack is unchanged: Next.js 16 (App Router), React 19, TypeScript and Tailwind on a Supabase backend (Auth, PostgreSQL with row-level security, Storage), deployed on Vercel. Five people work in parallel, each owning one feature folder. Every cross-feature contract (current user, progress recording, translations, quiz link, friends list) was frozen as a stub in a first "Wave 0". Features were then built in parallel (Wave 1) and integrated (Wave 2). Where the M1 idea met detailed design, we changed it:

- **Two numbers instead of one rating.** *Level* comes from XP, never drops, and gives a title. *Skill rating* is Elo (start 800, floor 100). It sets question difficulty, duel matchmaking and an XP multiplier, and only the student sees it. The quiz end screen shows the change in green or red.
- **Coins narrowed.** +3 per correct first try, +10 per daily login, and a 1-coin reattempt wager that is refunded if the reattempt is right. Coins buy only reattempts; duels and the placement quiz give none.
- **PDFs became manga chapters.** Pages are pre-rendered to 720 px JPEGs instead of shipping a PDF viewer (E1). Chapters are never locked; a "Recommended for you" bubble map sizes each chapter by its recommendation score.
- **Reel quiz gate.** Every 5 watched reels (skips do not count) unlock a 3-question quiz on those reels. Scrolling stays locked until the quiz is *finished*, and a stop card appears after 30 reels.
- **Duels.** 30 s rounds, five questions. The first correct answer to reach the server wins the round, and a wrong answer freezes that player. Postgres decides the order with a partial unique index. A bot opponent's answers are rolled on the server when the duel is created. For the demo, phones poll a duel-state endpoint every second instead of using Realtime.
- **Onboarding.** Language, grade and 3–4 persona questions, plus an optional placement quiz that seeds the skill rating.
- **XP and coin integrity** (new). A student can call any server action with any arguments, so the client never sends an amount, a correctness flag or a winner. Every award goes through one service-role database function into a ledger keyed by a unique `(user, ref)`, which makes repeated calls harmless.
- **Privacy.** We store only email, username, language and grade. Other students see a username and title. Leaderboards (Friends and Global, last-7-day XP) show no numbers. The forum blocks phone numbers and e-mail addresses and hides posts after reports.
- **Tooling.** Unit tests use Node's built-in `node --test` instead of Vitest/Jest, so there is no extra dependency.

2 Experiments and Results

E1 — Lesson delivery on low-end phones. The 13 chapter PDFs total 54.9 MB (4.2 MB per chapter). Rendered to 97 JPEG pages at 720 px and quality 70, they total 17.5 MB (180 KB per page), **3.1× smaller**. The reader also loads one page at a time, so the first page needs about 180 KB instead of the whole 4.2 MB file. Result: we use pre-rendered pages and no PDF.js bundle.

E2 — Question bank. 185 English questions: budget 45, save 45, invest 50, safety 45. By tier: bronze 60, silver 60, gold 20, platinum 25, diamond 20. An automated check validates every question's shape and tags, and checks that each reel has exactly one question and each chapter 5–8. It found the bank thin at the top tiers, so high-rated students will see repeats sooner.

E3 — Selection and recommendation behaviour. We ran the pure algorithms on fixed test data:

- A practice quiz gives 10 distinct questions, easiest first, at every rating, and the weakest pillar gets 4 of them.
- A missed question comes back once it is due.
- The reel feed keeps variety, puts a random pick in every 5th slot and allows one stretch item per page.
- Duel picks cover tiers below, at and above the player's own. The bot answers in 6–15 s with tier-based odds.

All behaved as specified.

E4 — Wave 2 integrity review. We reviewed the integrated code adversarially, as a student with a valid login would attack it:

- *Critical:* a database permission let a thread author rewrite other students' forum replies.
- *High:* a losing duel player could avoid the rating loss by closing the app.
- *High:* an empty reel quiz cleared the reel quiz gate.
- *High:* "answer accepted" XP could be farmed by moving the accept between replies.
- *High:* two migrations shared one version number.

All are scheduled as Wave 3a fixes.

3 Progress on Testing

Level	Status	Detail
Unit	48/48 pass	13 files, 0.16 s: rating maths, quiz scoring and coins, duel rounds and rewards, simulation replay, recommendations, reel feed and quiz gate, forum safety filter and XP caps, sign-up validation and routing, IST dates, question bank
Static	pass	`npm run build`, `tsc --noEmit` and `eslint` on the integrated Wave 2 code
Integration	partial	Placement, profile and dashboard checked against a mocked server. Migrations are not yet applied to Supabase
Security	review done	E4 findings scheduled. Per-table row-level-security tests wait for the live database
End-to-end, realtime	pending	Scripted demo path from sign-up to dashboard; a two-phone duel test
Performance, usability	pending	360 px screenshots, slow-3G video playback, reader on a low-end phone; NGO pilot with 30–50 students

4 Plan for the Remaining Time

1. **Wave 3a fixes** (all owners, in parallel): the E4 bugs first. Then make the server settle both duel players when a duel ends, stop bot duels from being cancelled for free, and stop quick-match queue rows from getting stuck. Move the forum daily caps into the database, and fix the reel quiz gate and the chapter-bonus edge cases.
2. **Deploy**: apply every migration to the Supabase project, set environment variables on Vercel for build and runtime, and deploy. Seed 6–8 demo students with XP history and friendships.
3. **Content**: Telugu and Hindi for all interface text and the question bank. Claude drafts each translation and a native speaker on the team checks it before it goes live. Replace the 12 placeholder reel clips and add more reels.
4. **Testing**: run the end-to-end demo path, the two-phone duel test, row-level-security checks, and 360 px and slow-3G checks. Then run a "teaches, never recommends" review of every question and chapter.
5. **Stretch** (only once the demo path works): an NGO coordinator cohort view, an installable app with an offline shell, classroom duel codes, badges and weekly goals.